AI usage

This project was developed with an AI assistant using **Copilot SDK in VS Code**.

What AI was used for

- Reading the user-supplied assignment and translating it into implementation requirements and verification checks.
- Researching WHO guidance, revised European AQI methodology, EEA data access, source schemas and reuse conditions.
- Designing and implementing the Python application, SQLite storage, ingestion, configurable references, UI, monitoring, portable packaging and documentation.
- Writing and running automated tests, real-data imports and performance checks.

Important corrections and decisions

- An initial React/PostgreSQL/Docker proposal was simplified to server-rendered HTML and SQLite after discussion with the owner.
- Copying a virtual environment was not treated as portable. The owner explicitly chose bundled Windows Python and packages so recipients need no Python install.
- The initial source assumption was corrected using real files: validated history is daily, current data hourly. The owner approved UTC dates to preserve matching averaging windows instead of pretending daily means could be shifted accurately.
- EEA dataset selectors and storage paths were checked rather than inferred. The current container is `airquality-p`; server date filters were observed to omit overlapping files, so filtering is performed locally.
- Hourly European AQI bands and annual WHO guidance were not applied to arbitrary historical period averages.
- Default interactive API documentation would have needed external assets and inline scripts; it was replaced with offline documentation and generated OpenAPI JSON.
- Test failures were investigated and corrected. For example, a coverage-boundary fixture originally requested a date beyond its seeded available history; it was corrected to test coverage without accidentally testing an unavailable period.
- Relocation checks exposed an offline-status issue: a cached successful import message could still describe online updates. Both server-rendered and client-refreshed status now use the current offline setting, with a regression test.
- A later request added an explicit stop launcher for manual ZIP creation. Cooperative shutdown, scoped leftover-process cleanup, SQLite release checks and preservation of unrelated Python processes were exercised on Windows.

Workflow and responsibility

The assistant researched primary sources, implemented incremental milestones, ran real-data checks and automated tests, measured latency and tested relocation. The owner approved portable Windows packaging, UTC day semantics and local Git milestone commits. Initial implementation and verification were completed locally, before the later public-publication request.

Public publication was subsequently explicitly requested by the owner: one source snapshot commit at Clean-Air-Project, plus the complete portable application as a GitHub Release. Original development history is retained on a local backup branch. Public release packages omit internal Git metadata, live logs, credentials and temporary state.

The owner subsequently requested that the portable runtime and database also appear in the public `main` branch. They are tracked with Git LFS where needed; the owner approved replacing the published snapshot to keep `main` at one commit. The earlier public snapshot and development milestones remain preserved locally, and the existing portable Release remains available.

The owner then requested publication of the complete runnable project at Clean-Air-Repository. That checkout includes the source, bundled runtime, Python packages and database snapshot, with large binaries and the database stored through Git LFS. Logs, credentials, caches and other transient local state remain excluded.

Synthetic measurements exist only in automated test fixtures. The application does not substitute mock data when a source fails.

The AI-generated implementation and automated checks are not an independent human review. The submitting developer remains responsible for understanding the methodology, verifying the results, reviewing changes and being able to explain and extend every submitted component during the technical interview.